

In a distributed banking environment, customers access their accounts and conduct transactions (deposits, withdrawals, and queries) across multiple branches of the bank. Each branch operates as an independent replica with a local balance, which must remain consistent with the balances at other branches. The goal of this project is to implement a client-centric consistency model in a distributed banking system to ensure consistent behavior for individual clients interacting with replicated branches. Specifically, the project focuses on the Read Your Writes consistency, guaranteeing that any write operation (e.g., deposit or withdrawal) performed by a customer is reflected in subsequent read operations (e.g., querying account balance), regardless of which replica the client interacts with. This addresses issues where clients observe outdated data due to delays in propagating updates between replicas.

To set up the environment to implement this project, we had to use python's grpcio (version 1.67.0) and grpcio-tools (version 1.67.0) packages. The protobuf version accompanying the grpcio-tools was version 5.28.3. I additionally downloaded a protobuf 3 extension to my vscode setup for syntax validation. I then started implementing the project by defining the services needed to support the interactions between customer and branch, and branch and branch.

To support the messages, I defined an interface enum (it defined the types of transactions), and a result enum. The interface enum had fields WITHDRAW, DEPOSIT, and QUERY. The result enum had fields SUCCESS and FAILURE. I then defined messages CustomerMessage, BranchMessage, and Reply. CustomerMessage are messages intended only to be sent by customers to a branch. It consisted of the fields id (for customer id), request_id (id for a customer request), interface (type of transaction a customer wished to perform), amount (even though all fields are optional by default in a proto message, I wanted to clearly outline that amount is optional as it is only used in tandem with WITHDRAW and DEPOSIT interfaces) and writeset (a list of integers representing the customer's write request ids, only set when the interface is WITHDRAW or DEPOSIT). BranchMessage are messages intended only to be sent between branches. It consisted of the fields interface (type of transaction to be performed at a branch), amount, and writeset (validated writeset propagated by origin branch). amount is not optional in this message because the gRPC services are designed such that banks only communicate with each other to propagate withdraw or deposit transactions. The Reply message is a message used to reply to both customers and branches. It consists of the fields interface, result (indicates whether a request was successful or not), and balance (optional, is only returned when the interface is query).

The two services I defined were Query and Propagate. Query is intended for customer to branch communication (customer sends a query request with CustomerMessage as argument and expects a Reply message). Propagate is intended for branch to branch communication (branch sends a propagate request with BranchMessage as argument and expects a Reply message).

To be able to support customer requests, the Branch class defined in branch.py has `_query`, `_withdraw`, `_deposit`, and `_check_writeset` methods as well as `Query`. When a branch object receives a `Query` rpc, it first validates the request's writeset by comparing it to its own with the `_check_writeset` method. Validation is only performed for write operations in the method. It then checks the interface of the `CustomerMessage` received. If it is `QUERY`, it creates a `Reply` message consisting of the interface initially received, the result of the request, and the balance of the branch. If the interface is either `WITHDRAW` or `DEPOSIT`, the branch propagates these requests to the other branches by first establishing communication channels to the list of branches the current branch is aware of (including itself) and sending a `Propagate` request to all those branches. When a branch receives a `Propagate` RPC, it credits or debits (depending on the interface) its balance as per the amount in the `BranchMessage`. It then increments its writeset with the customer request id sent in the request before responding with a `Reply` message. If at least one of the responses is a failure, `_withdraw` and `_deposit` would return false and we would construct a `Reply` response with its result set to `Failure`. (nb. Since this is a simplified model, proper transaction semantics were not implemented. In the case of a proper model, all requests completed prior to the failure would have to be reversed.) If all communications with other branches in a branch's list of branches are successful, `_withdraw` and `_deposit` would return true and we would construct a `Reply` response consisting of the request interface and the result of the RPC call (successful). If the interface of a request is not any of the three discussed, we respond with a failed `Reply` message.

The customer class defined in customer.py consists of the methods `createStub`, `executeEvents`, and `returnMsgs`. `createStub` establishes a communication channel to a branch on a server in another process. All branches are on processes connecting to port `50,000+branch_id` (this is to ensure that the ports used are very unlikely to be used by other processes when the servers are initially set up). Thus if a customer wants to communicate with a branch, it can do so on port `50,000+branch_id`. `executeEvents` iterates over the list of events a customer ought to perform (this is stored in `events` as a list of dictionaries). For each event, I first instantiate the communication channel with the `branch_id` of the event and then perform an RPC to the branch depending on the interface of the event. After this, I construct the outcome of the event depending on the interface and then store it in the `recvMsg` class field before closing the communication channel.

To run the servers, the json file containing the input is first read. From there, each branch's set up information and a list of all branch ids are retrieved. When a branch server is created, we first create a `Branch` object with the branch id, the starting balance, and the ids of the branches it can communicate with. A gRPC server is then instantiated and we add our branch service as a servicer. The server is then connected to a port which it can listen to requests from before starting the server. I used python's multiprocessing module to instantiate each server on a different process and have it listen on ports according to their id (all branches are on processes connecting to port `50,000+id`).

To run the client, the json file containing the input is read in. From there, we retrieve each client's information and return a list of Customer objects using the information we retrieved. I ran the clients in sequential order on a single process. For each customer object, I called the executeEvents method and stored the results to be written to the output json file. In the given input file, only one customer is present.

The results stored in the output json file consist of the id of the communicating customer and the responses received for each action then performed. The actions performed follow a read-write-read sequence to check if the propagation of each write operation (withdraw or deposit) is successful. The output is a list containing the customer id and the list of events it processed. The events contain information on the query interface, the branch the customer was communicating to, and the balance if the request was a query or the result if the request was a write operation. This list is in the same order of read-write-read, with the first read reflecting the last write operation, if any, and each second read operation reflecting the update of the write operation in the sequence. This displays conformity with the Read Your Writes policy.